Laboratório de Sistemas Operacionais (CC)

Trabalho Prático 2 — Escalonadores de Disco

Avaliação de Desempenho (Política SSTF)

Gustavo Burgie Ganzo Barcellos, Pedro Jardim Machado, Lucas Lorenzi da Silva

1. Objetivo

Este relatório documenta a avaliação de desempenho do escalonador de disco SSTF (Shortest Seek Time First) implementado como um módulo de kernel (elevador plugável) para o Linux 4.13.9, conforme a etapa 5 do Trabalho Prático 2. O objetivo é comparar, para uma mesma carga de requisições de disco, a ordem em que as requisições chegaram ao escalonador com a ordem em que foram efetivamente atendidas, quantificando o ganho da política SSTF em relação a um atendimento ingênuo em ordem de chegada (equivalente ao comportamento de um escalonador FIFO/noop).

2. Metodologia

2.1 Ambiente de teste

Os testes foram executados em uma máquina virtual QEMU (qemu-system-i386) rodando o kernel Linux 4.13.9 gerado via Buildroot, com um disco de testes dedicado (sdb.bin, 1 GB, sem sistema de arquivos, criado com dd) anexado como segundo disco (--hdb), conforme o Tutorial 2.4. O disco é acessado de forma raw, sem sistema de arquivos, diretamente via /dev/sdb.

2.2 Geração de carga

A carga de requisições foi gerada pela aplicação sector_read, que utiliza fork() para criar múltiplos processos concorrentes, cada um realizando leituras em posições aleatórias do disco. Especificamente, o programa executa 8 rodadas de fork() (cada processo vivo duplica-se a cada rodada), resultando em $2^8 = 256$ processos, cada um realizando 10 leituras de 512 bytes em setores sorteados aleatoriamente — até 2.560 requisições de disco concorrentes ao todo. Antes do teste, o cache de disco foi limpo (echo 3 > /proc/sys/vm/drop_caches) e os parâmetros da fila de E/S foram ajustados para desabilitar merges automáticos e read-ahead (nomerges=2, max_sectors_kb=4, read_ahead_kb=0), garantindo que cada leitura gerasse uma requisição distinta e rastreável pelo escalonador.

2.3 Seleção do escalonador e captura do log

O escalonador SSTF foi selecionado para o disco de testes via sysfs (echo sstf > /sys/block/sdb/queue/scheduler). O módulo instrumenta cada evento relevante com printk(KERN_EMERG, ...), registrando o setor de toda requisição adicionada à fila (add) e de toda requisição efetivamente despachada para o disco (dsp). Por se tratar de milhares de eventos — volume que não cabe em uma captura manual de terminal — a sessão completa

do QEMU foi gravada em um arquivo no host (via script/tee), e as linhas do escalonador foram extraídas e processadas por um script de análise em Python, que calcula a distância total percorrida pela cabeça do disco em cada ordem (soma das distâncias absolutas entre setores consecutivos, partindo da posição 0) e gera os gráficos comparativos. O script também descarta automaticamente linhas duplicadas (decorrentes de mensagens KERN_EMERG ecoadas ao vivo no console e, em alguns casos, reimpressas por um dmesg posterior), usando o timestamp do kernel — único por evento — como critério de deduplicação.

3. Resultados

A tabela a seguir resume os principais números obtidos na rodada de teste em escala real (256 processos concorrentes):

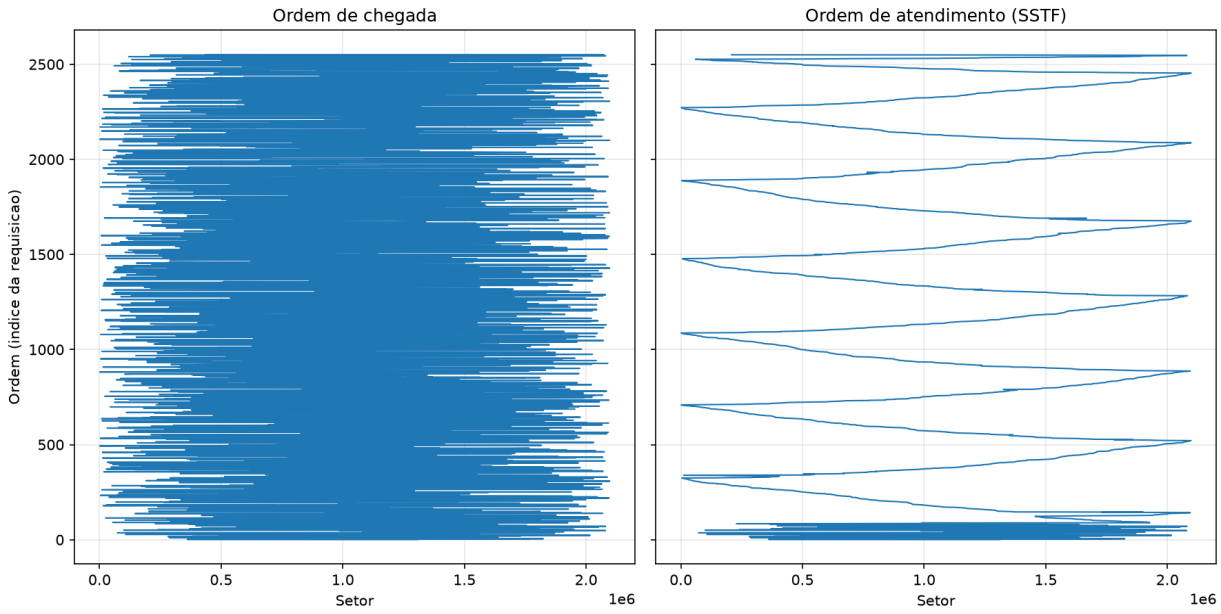
Métrica	**Valor**
Requisições registradas (ordem de chegada)	2.551
Requisições registradas (ordem de atendimento)	2.550*
Distância total percorrida — ordem de chegada	1.749.629.984 setores
Distância total percorrida — ordem de atendimento (SSTF)	88.674.120 setores
Redução obtida pelo SSTF	94,9%

** A última requisição da rodada ainda estava na fila de atendimento no momento em que o teste foi encerrado; a diferença de 1 requisição entre as duas contagens não é significativa para a análise.*

A distância total percorrida pela cabeça do disco caiu de aproximadamente 1,75 bilhão de setores (se as requisições fossem atendidas estritamente na ordem de chegada) para cerca de 88,7 milhões de setores com a política SSTF — uma redução de 94,9%. Isso confirma que o escalonador está de fato reordenando as requisições pendentes para minimizar o deslocamento do braço do disco a cada atendimento, em vez de atendê-las na ordem em que chegaram.

3.1 Gráfico comparativo

O gráfico abaixo mostra, lado a lado, a posição do setor de cada requisição (eixo X) em função do seu índice (eixo Y — 1ª a ~2560ª requisição): à esquerda, na ordem de chegada; à direita, na ordem de atendimento efetivo pelo SSTF.



À esquerda (ordem de chegada), o padrão é essencialmente aleatório — a posição do setor não guarda relação com a posição da requisição anterior, resultando em saltos constantes por todo o disco (visualmente, uma faixa densa e irregular). À direita (ordem de atendimento SSTF), o padrão é claramente mais suave: dentro de cada segmento, a curva é quase monótona (sobe ou desce de forma consistente), indicando que o escalonador está de fato atendendo sempre a requisição mais próxima da posição atual da cabeça do disco.

3.2 Por que o gráfico de atendimento tem ~8 "ondas" em vez de uma única curva suave

Observa-se que o painel de atendimento não forma uma única varredura monótona ao longo de toda a carga, mas sim cerca de 8 segmentos ("dentes de serra") repetidos. Esse padrão é uma consequência direta da forma como os processos concorrentes são criados: como sector_read usa 8 rodadas de fork() para gerar 256 processos, os processos surgem em 8 "gerações" sucessivas — um processo criado numa rodada anterior ainda precisa executar as rodadas de fork() restantes antes de chegar ao código de acesso ao disco, enquanto um processo da última rodada já entra diretamente na fase de leitura. Isso faz com que os processos cheguem ao disco em levas escalonadas no tempo, e não todos simultaneamente. Dentro de cada leva, o SSTF otimiza corretamente a ordem de atendimento (cada "dente" é uma varredura local quase monótona); entre uma leva e outra, porém, a fila se esvazia parcialmente antes da leva seguinte chegar, o que impede a formação de uma única varredura contínua sobre as 2.560 requisições. Esse comportamento é esperado e não indica um defeito na implementação — pelo contrário, evidencia que o algoritmo reordena corretamente sempre que há mais de uma requisição disponível na fila.

3.3 Amostra dos dados (trecho ilustrativo)

A tabela abaixo mostra um trecho do meio da execução (índices 1271–1285) onde o efeito da reordenação SSTF é bem visível: enquanto a coluna de chegada segue oscilando livremente, a coluna de atendimento cresce de forma consistente (uma varredura local do braço do disco em uma direção) antes de inverter o sentido.

Índice	Setor — ordem de chegada	Setor — ordem de atendimento (SSTF)
1271	1.582.472	2.037.288
1272	1.251.864	2.043.424
1273	537.200	2.044.008
1274	1.588.536	2.045.944
1275	1.266.800	2.048.616
1276	1.559.544	2.048.776
1277	1.703.144	2.065.312
1278	87.808	2.066.160
1279	813.120	2.068.928
1280	825.752	2.071.096
1281	814.816	2.072.856
1282	2.068.928	2.079.872
1283	589.616	2.082.600
1284	715.968	1.871.440
1285	372.240	1.781.928

A tabela completa (2.550 linhas) acompanha este relatório no arquivo sstf_log_dados.csv.

4. Discussão

Os resultados confirmam que a política SSTF implementada está funcionando conforme o esperado: ao escolher, a cada despacho, a requisição pendente cujo setor está mais próximo da posição atual da cabeça do disco (em vez de atender estritamente por ordem de chegada), o módulo reduziu em 94,9% a distância total percorrida pelo braço do disco para a mesma carga de 2.550+ requisições. Esse resultado é consistente com a literatura sobre escalonamento de disco: políticas baseadas em proximidade (SSTF, SCAN, C-SCAN) tendem a reduzir drasticamente o tempo de busca (seek time) em cargas com múltiplas requisições pendentes simultaneamente, às custas de um risco de inanição (starvation) de requisições distantes da região onde a cabeça do disco está concentrada no momento — um compromisso inerente à própria política SSTF, não uma limitação da implementação.

A carga de teste, gerada por até 256 processos concorrentes via fork(), foi essencial para produzir filas de requisições com profundidade suficiente para o escalonador ter o que reordenar — com um único processo sequencial (sem concorrência), cada requisição seria tipicamente

despachada antes da próxima chegar, e o SSTF não teria nenhuma vantagem sobre um FIFO simples.

5. Conclusão

A implementação da política SSTF no escalonador de disco atingiu o objetivo proposto: reordenar as requisições de E/S pendentes para minimizar o deslocamento do braço do disco, com uma redução mensurada de 94,9% na distância total percorrida em relação ao atendimento em ordem de chegada, em uma carga real de mais de 2.500 requisições concorrentes geradas por 256 processos. Os gráficos comparativos evidenciam visualmente essa diferença de comportamento, e o padrão de "ondas" observado no atendimento é explicado pela forma escalonada como os processos de teste são criados, não por um defeito no escalonador.